DURS: A Distributed Method for k-Nearest Neighbor Search on Uncertain Graphs

Xiaodong Li
Department of Computer Science
The University of Hong Kong
Hong Kong, China
xdli@cs.hku.hk

***Abstract*—Large graphs are increasingly prevalent in mobile networks, social networks, traffic networks and biological networks. These graphs are often uncertain, where edges are augmented with probabilities that indicates the chance to exist. Recently *k*-nearest neighbor search has been studied within the field of uncertain graphs, but the scalability and efficiency issues are not well solved. Moreover, solutions are implemented on a single machine and thus cannot fit large uncertain graphs. In this paper, we develop a framework, called DURS, to distribute *k*-nearest neighbor search into several machines and re-partition the uncertain graphs to balance the work loads and reduce the communication costs. Evaluation results show that DURS is essential to make the system scalable when answering *k*-nearest neighbor queries on uncertain graphs.**

***Index Terms*—*k*-nearest neighbor, uncertain graphs, distributed system, scalable search**

I. Introduction

Nowadays social networks, traffic networks, biological networks, and mobile ad-hoc networks are generating larger graphs [1], but due to noisy measurements, hardware limitation, inference models [2], and privacy-preserving perturbation, these graphs are inherently uncertain.

Uncertain graphs. To model this uncertainty, *uncertain graphs* have been studied [3], where each edge is associated with a probability p . Figure 1(a) shows a mobile network as an uncertain graph $\mathcal{G}$, where each node denotes a mobile phone, and each edge is a channel. The value on each edge denotes the probability that the channel exists. For instance, the existential probability of channel (D, F) is 0.45 and thus can be considered as a weak one.

Possible worlds. Given an uncertain graph $\mathcal{G}$, there are several possible instances. For example, G_1 (i.e. Figure 1(b)) is a possible instance sampled from $\mathcal{G}$ (i.e. Figure 1(a)), where only five channels exist in the mobile network. These instances are called *possible worlds*. As described in Definition 1, the probability of each possible world can be calculated, e.g., $Pr(G_1) = 0.0042$. There are $2^{|\mathcal{G}.E|}$ possible worlds, making the queries challenging on uncertain graphs.

k-NN query. In this paper, we study k-nearest neighbor (k-NN) queries on uncertain graphs. The k-NN query is important in tasks like protein complex detection, link discovery and function prediction. Given a node s, the k-NN query will return k nodes with the shortest *distances*[1] from s. It is formally defined in Definition 2 [3].

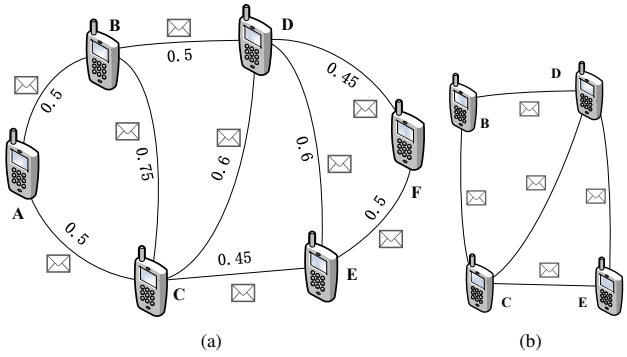


Fig. 1. (a) Uncertain graph $\mathcal{G}$ with packet delivery probability in mobile networks and (b) a possible world G_1 from $\mathcal{G}$ with existential probability $Pr(G_1) = 0.5 \times 0.5 \times 0.75 \times 0.5 \times 0.6 \times 0.45 \times 0.6 \times 0.55 \times 0.5 = 0.0042$.

Definition 1: Given an uncertain graph $\mathcal{G}$=(V, E, p), the probability of observing a possible world G=(V, E_G) is

$$Pr(G) = \prod_{e \in E_G \& i \in W \setminus \{0\}} p(i|e) \prod_{e \in E \setminus E_G} p(0|e). \quad (1)$$

Definition 2: Given $\mathcal{G}(V, E, p)$, a source node $s \in V$, and an integer $k > 0$, k-NN returns a set of nodes C, $|C| = k$:

$$C = \{t | d(s,t) \leq d(s,p), \forall p \in V \setminus C\}, \ where \quad (2)$$

$$d(s,t) = 1 - \sum_{0 \leq a < \infty} \sum_{d_G(s,t)=a} Pr(G). \quad (3)$$

Scalability issue. To answer the k-NN query on uncertain graphs, every possible world needs to be sampled and searched, and then all these answers are collected to form the final answer. Many methods are proposed to speed up this process, but the scalability and efficiency issues are not well solved [3], especially for the large uncertain graphs. We call a system scalable if the response time remains small (e.g., a few seconds) when increasing the workloads. An improperly designed cluster will make some workers busy but others

Xiaodong Li is a PhD student in the Department of Computer Science, University of Hong Kong, and supervised by Dr. Reynold Cheng.

[1]In the field of uncertain graphs, there are several distance functions. Here we focus on *reliability* (i.e. Equation 3). Other distance functions can also be used in the framework.

2375-0324/19/$31.00 ©2019 IEEE
DOI 10.1109/MDM.2019.00-23

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on January 29,2024 at 11:53:25 UTC from IEEE Xplore. Restrictions apply.

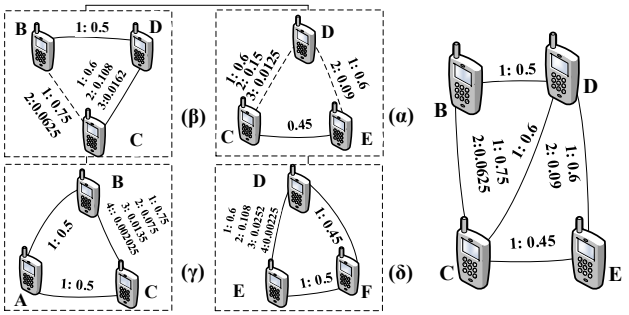


Fig. 2. (a) U-tree for the uncertain graph in Figure 1(a) (b) the uncertain subgraph for $d(B, E)$.

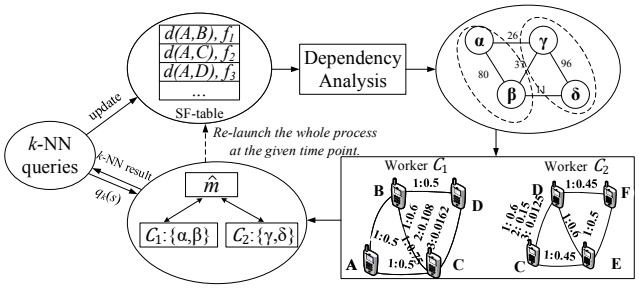


Fig. 3. Flowchart of DURS with U-tree re-partition.

totally idle, or too high communication cost, leading to skew and inefficiency. In these cases, adding new machines will not increase the performance. So we propose DURS, a Dynamic U-tree Re-partitioning Scheme to relieve the correlation and balance the workloads in the cluster to ensure the scalability and efficiency.

II. Dynamic U-tree Re-partition Scheme

In our preliminary work [3], an indexing framework called U-tree, is developed to support DURS. For example, Figure 2(a) is built for uncertain graph $\mathcal{G}$ in Figure 1(a). Then each $d(s,t)$ can be answered by querying U-tree. For example, an uncertain subgraph $\mathcal{G}_{\{\alpha,\beta\}}$ (i.e. Figure 2(b)) will be generated to answer $d(B,E)$. Query time will be saved because $|\mathcal{G}_{\{\alpha,\beta\}}.E| < |\mathcal{G}.E|$. Fewer possible worlds will be sampled according to Definition 1.

U-tree can be deployed into several machines by assigning its bags (e.g., $\alpha, \beta, \gamma, \delta$ in Figure 2(a)) to different machines. This is because U-tree has lower correlations among the bags, e.g., each U-tree bag has lower dependency on the other bags compared to other indexes [3]. Also, we develop a method to quantify the correlations among U-tree bags with the help of users query log, called SF-table. For example, if $d(A,D)$ is queried f_3 times in the system, then we can infer bags (β,γ) are searched together for f_3 times and thus the pair-wise correlations are quantified.

We show the work flow of DURS in Figure 3. After accessing the current SF-table, we get the frequency of each $d(s,t)$. According to Definition 2, each incoming k-NN query will update $|\mathcal{G}.V|-1$ source-to-target queries in the SF-table. Then we launch dependency analysis to calculate the pair-wise correlations among the U-tree bags. Then we employ spectral clustering to split U-tree bags into several partitions.

Within each partition, inner cohesiveness is achieved (i.e., high correlations among the U-tree bags), thus communication overhead is reduced; across different partitions, inter incoherence is achieved (i.e., low correlations among the U-tree bags), thus the workloads among different machines are balanced, since hotspot data are separated into different machines in this manner. A parameter θ is designed to control the maximum percentage of $\mathcal{G}$ to be contained in a node, thus balancing the above two ends. A bigger θ means DURS trends to allow each machine to contain more bags, thus reducing more communication overhead; a smaller θ means DURS trends to split the big partitions into smaller ones to balance the workloads, thus giving more confidence on the cluster scalability.

TABLE I
DURS performance by tuning θ.

θ	1	0.8	0.6	0.4	0.2
Query time(Sec)	13	8	39	41	40
idle threads	16	8	20	24	24

The repartition scheme above will be launched at a given time interval, e.g., every six hours. It helps DURS capture the query trends and develop the cluster plan accordingly. For example, in a traffic network with uncertainty, users may trend to query the business areas in the daytime, thus these parts of U-tree bags become popular and should use more machines; however, the condition at night will be very different, thus a re-partition scheme is essential. When a k-NN query comes, DURS will assign the source-to-target queries $d(s,t)$ to the corresponding machines and collect the distance values.

We run experiments on a 8-node Galaxy cluster, each with a 4-core Intel i7-3770 3.40GHz processor and 16GB of memory. Spark 2.2.0 is deployed as the running environment. We model Hong Kong traffic network with 100K edges as an uncertain graph. On each edge (i.e. road segment), there is a probability distribution of passing times from analyzing 323M Hong Kong taxi trajectories. As shown in Table I, best performance is achieved when $\theta = 0.8$. Since $\theta = 1$ is actually a single-machine solution, it can serve as a baseline, which shows improper partitioning may lead to lower performance because of higher communication cost.

References

[1] Y. Fang, Z. Wang, R. Cheng, X. Li, S. Luo, J. Hu, and X. Chen, "On spatial-aware community search," *IEEE TKDE*.
[2] Y. Fang, R. Cheng, X. Li, S. Luo, and J. Hu, "Effective community search over large spatial graphs," *VLDB Endowment*, 2017.
[3] X. Li, R. Cheng, Y. Fang, J. Hu, and S. Maniu, "Scalable evaluation of k-nn queries on large uncertain graphs," *EDBT*, 2018.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on January 29,2024 at 11:53:25 UTC from IEEE Xplore. Restrictions apply.